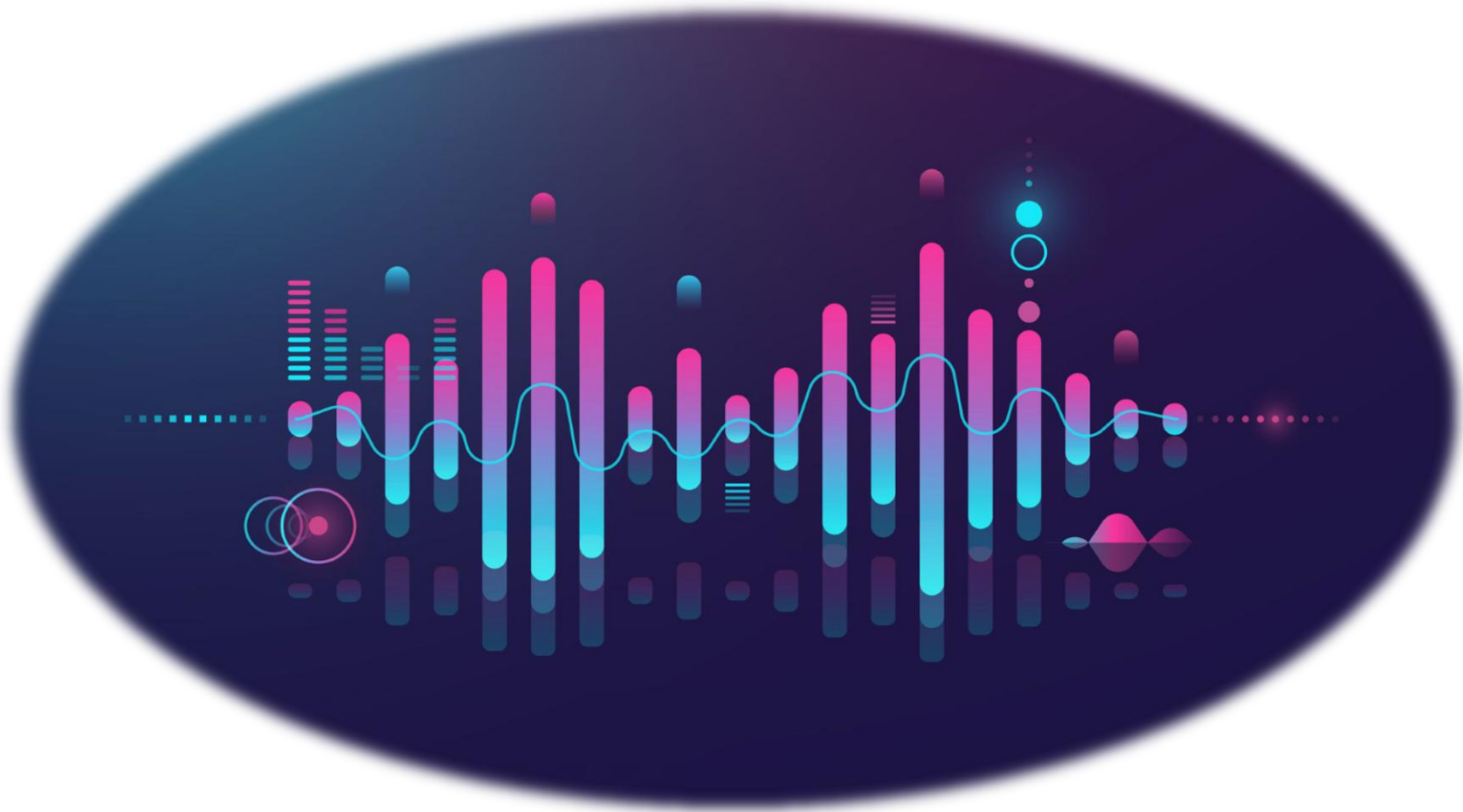


Classificazione generi musicali del dataset “FMA-small”



Obiettivo

- L'obiettivo principale di questo progetto è quello di sviluppare modelli in grado di distinguere tra diversi generi musicali basandosi su dati audio, ossia **classificare automaticamente i generi musicali** usufruendo di diverse tecniche di machine learning.

Modelli implementati

- I modelli realizzati includono:
 1. Una **rete neurale convoluzionale (CNN)**;
 2. Una **rete neurale feedforward (FNN)**;
 3. Una **Support Vector Machine (SVM)**.

Dataset utilizzato

- Il progetto sfrutta il dataset **FMA (Free Music Archive)**, un insieme di file audio di alta qualità, e tecniche di estrazione di caratteristiche audio avanzate;
- Questo dataset è una collezione di brani musicali di dominio pubblico che copre vari generi musicali, fornendo una base ideale per la classificazione automatica degli stessi.

Caratteristiche del dataset

- I file audio sono forniti in formato **MP3** con una frequenza di campionamento di **44.1 kHz** e una durata media di **30 secondi** per brano;
- Il dataset FMA è disponibile in diverse dimensioni, ma per questo progetto è stata utilizzata la versione **FMA-Small**, che contiene **8.000 brani** distribuiti su **8 generi** diversi. Questi ultimi vengono ricavati dal file “**tracks.csv**”.

Imperfezioni di alcuni file audio

- Il dataset **FMA-Small** presenta file audio con alcune imperfezioni legate alla durata e alla riproducibilità degli stessi sul PC:
 - I file audio chiamati “099134”, “108925” e “133297”, facenti parte del dataset di training, non possono essere caricati nel suddetto dataset e non possono essere riprodotti;

99134	training	small
108925	training	small
133297	training	small

Imperfezioni di alcuni file audio

2. I file audio di nome “098567”, “098569” e “098565”, del dataset di training, hanno rispettivamente durata di 3, 5 e 15 secondi, inferiore a quella prevista di 30 secondi, ma sono caricabili nel dataset;
3. 1212 brani hanno una durata di 29 secondi, però, come i precedenti, sono caricabili senza errori nei vari dataset.

```
98565    training    small
98567    training    small
98569    training    small
```

Generi musicali da classificare

- I **generi musicali** da classificare sono:

1. Electronic
2. Experimental
3. Folk
4. Hip-Hop
5. Instrumental
6. International
7. Pop
8. Rock

File “base_config.yaml”

- Il file **“base_config.yaml”** è un file di configurazione che definisce tutte le impostazioni necessarie per il processo di addestramento di un modello di machine learning per la classificazione di audio;
- Questo file rende il processo di sviluppo del modello più efficiente, flessibile, e meno incline a errori, migliorando al contempo la riproducibilità e la gestione degli esperimenti.

File “base_config.yaml”

- È formato da sei sezioni:
 1. **best;**
 2. **data;**
 3. **filename;**
 4. **model;**
 5. **split;**
 6. **training.**

```
best:
  cnn: best_cnn_model.pt
  ff: best_fnn_model.pt
  svm: best_svm_model.pkl

data:
  dataset_dir: fma_small_data/fma_small
  metadata_file: fma_small_data/tracks.csv

filename:
  test_embeddings: test_embeddings.npy
  test_labels: test_labels.npy
  train_embeddings: training_embeddings.npy
  train_labels: training_labels.npy
  val_embeddings: validation_embeddings.npy
  val_labels: validation_labels.npy

model:
  dropout: 0.3
  hidden_layers: [384, 256, 192, 128, 96, 64, 48, 32, 24, 16]
  input_size: 768
  num_classes: 8

split:
  test: test
  train: training
  val: validation

training:
  batch_size: 32
  best_metric_lower_is_better: false
  checkpoint_dir: checkpoints/
  device: cuda
  epochs: 100
  evaluation_metric: accuracy
  learning_rate: 0.001
  max_duration: 30
  npy_dir: npy_files/
  optimizer: adam
  sample_rate: 16000
  warmup_ratio: 0.1
  weight_decay: 0.001
```

Classe “FMADataset”

- La **classe “FMADataset”** rappresenta la struttura dati principale utilizzata nel progetto per la gestione e l'analisi del dataset Free Music Archive (FMA);
- È costituita da tre metodi:
 1. **__init__**;
 2. **__len__**;
 3. **__getitem__**.

```

import numpy as np
import os
import pandas as pd
import sys
import torch

from torch.utils.data import Dataset

# Append the parent directory to the system path to enable relative imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
from utils import get_audio_path, get_genre_idx, get_track_id, load_audio, process_audio

class FMADataset(Dataset):
    def __init__(self, root, metadata_file, data_split, sample_rate, max_duration)::
        self.root = root # Path to the root directory of audio files
        df = pd.read_csv(metadata_file, low_memory=False) # Load metadata from the CSV file
        self.metadata = df[(df['set'] == data_split) & (df['set.1'] == 'small')] # Filter metadata based on the data split and subset ('small')
        self.sample_rate = sample_rate # Desired sample rate for audio files
        self.max_length = sample_rate * max_duration # Maximum length of audio in samples, based on the duration

        # Get unique genres and create a mapping from genre name to an index
        self.genres = np.sort(self.metadata['track.7'].unique())
        self.genre_to_idx = {genre: idx for idx, genre in enumerate(self.genres)}

    def __len__(self):
        return len(self.metadata)

    def __getitem__(self, idx):
        # Get the track ID, audio file path, and genre index for the given sample index
        track_id = get_track_id(self.metadata, idx)
        audio_path = get_audio_path(self.root, track_id)
        genre_idx = get_genre_idx(self.metadata, self.genre_to_idx, idx)

        # Load the audio file and handle any errors that occur during loading
        audio = load_audio(audio_path, self.sample_rate) # During loading, converts the audio to mono and resamples to the sample_rate value
        if audio is None: # If there was an error loading the audio, try the next sample
            return self.__getitem__((idx + 1) % len(self.metadata))

        audio = process_audio(audio, self.max_length) # Process the audio: truncate or pad to the desired length
        return audio, genre_idx # Return the processed audio and its corresponding genre index

```

Classe “AudioEmbeddings”

- La classe “**AudioEmbeddings**” è progettata per estrarre **embedding**, vale a dire rappresentazioni numeriche, da audio utilizzando modelli pre-addestrati;
- È formata da due metodi:
 1. **__init__**;
 2. **extract**.

```

from transformers import AutoFeatureExtractor, AutoModel
import torch

class AudioEmbeddings:
    """
    This class is intended to extract embeddings from audio models.
    It uses Wav2Vec2 as a default model.
    """

    def __init__(self, model_name='ALM/hubert-base-audioset',
                  device=torch.device('cuda' if torch.cuda.is_available() else 'cpu')):

        self.processor = AutoFeatureExtractor.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)

        self.device = device
        self.model.to(self.device)

        self.model_name = model_name

        # eval mode
        self.model.eval()

    def extract(self, speech, sampling_rate=16000):
        """
        Extract embeddings from a speech.

        Args:
            speech (str): Speech to extract embeddings from.
            sampling_rate (int): Sampling rate of the input audio.

        Returns:
            torch.Tensor: Embeddings of the speech.
        """

        inputs = self.processor(speech, sampling_rate=sampling_rate,
                                return_tensors="pt", padding="longest")
        inputs = {name: tensor.to(self.device) for name, tensor in inputs.items()}

        with torch.no_grad():
            outputs = self.model(**inputs)

        return outputs.last_hidden_state.mean(dim=1).detach().cpu().numpy()

```

Classe “CNNAudioClassifier”

- La classe “**CNNAudioClassifier**” crea un classificatore audio fondato su una rete neurale convoluzionale (CNN);
- È composta da due metodi:
 1. **__init__**;
 2. **forward**.

```
import torch
import torch.nn as nn

class CNNAudioClassifier(nn.Module):
    def __init__(self, input_channels, num_classes, dropout):
        super(CNNAudioClassifier, self).__init__()

        # Layer 1: convolutional layer with 8 filters
        self.conv1 = nn.Conv1d(input_channels, 8, 3, stride=2, padding=1)
        self.bn1 = nn.BatchNorm1d(8)

        # Layer 2: convolutional layer with 16 filters
        self.conv2 = nn.Conv1d(8, 16, 3, stride=2, padding=1)
        self.bn2 = nn.BatchNorm1d(16)

        # Layer 3: convolutional layer with 32 filters
        self.conv3 = nn.Conv1d(16, 32, 3, stride=2, padding=1)
        self.bn3 = nn.BatchNorm1d(32)

        # Layer 4: convolutional layer with 64 filters
        self.conv4 = nn.Conv1d(32, 64, 3, stride=2, padding=1)
        self.bn4 = nn.BatchNorm1d(64)

        # Layer 5: convolutional layer with 128 filters
        self.conv5 = nn.Conv1d(64, 128, 3, stride=2, padding=1)
        self.bn5 = nn.BatchNorm1d(128)

        # GELU activation function: provides a good balance between sparsity and smoothness
        self.gelu = nn.GELU()

        # Dropout layer: helps prevent overfitting by randomly dropping units during training
        self.dropout = nn.Dropout(dropout)

        # Fully connected layers: for classification after the convolutional layers
        self.fc1 = nn.Linear(128, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 16)
        self.fc4 = nn.Linear(16, num_classes)
```

```
def forward(self, x):
    # Adds a new dimension at index 1 and swaps dimensions 1 and 2 of the tensor
    x = x.unsqueeze(1).transpose(1, 2)

    # Generates a tensor of random values from a normal distribution and adds randomness to the data
    x = x + torch.randn_like(x) * 0.1

    # Forward pass through the first convolutional layer + batch normalization + GELU activation
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.gelu(x)

    # Forward pass through the second convolutional layer + batch normalization + GELU activation
    x = self.conv2(x)
    x = self.bn2(x)
    x = self.gelu(x)

    # Forward pass through the third convolutional layer + batch normalization + GELU activation
    x = self.conv3(x)
    x = self.bn3(x)
    x = self.gelu(x)

    # Forward pass through the fourth convolutional layer + batch normalization + GELU activation
    x = self.conv4(x)
    x = self.bn4(x)
    x = self.gelu(x)

    # Forward pass through the fifth convolutional layer + batch normalization + GELU activation
    x = self.conv5(x)
    x = self.bn5(x)
    x = self.gelu(x)

    # Flatten the tensor for the fully connected layers
    x = x.view(x.size(0), -1)

    # First fully connected layer + dropout + GELU activation
    x = self.fc1(x)
    x = self.dropout(x)
    x = self.gelu(x)

    # Second fully connected layer + dropout + GELU activation
    x = self.fc2(x)
    x = self.dropout(x)
    x = self.gelu(x)

    # Third fully connected layer + dropout + GELU activation
    x = self.fc3(x)
    x = self.dropout(x)
    x = self.gelu(x)

    # Output layer, no activation here
    x = self.fc4(x)

    return x
```

Classe “FNNAudioClassifier”

- La classe **“FNNAudioClassifier”** fornisce un classificatore audio basato su una rete neurale feedforward (FNN);
- È costituita da due metodi:
 1. **__init__**;
 2. **forward**.

```
import torch
import torch.nn as nn

class FNNAudioClassifier(nn.Module):
    def __init__(self, input_size, hidden_layers, num_classes, dropout):
        super(FNNAudioClassifier, self).__init__()

        # Layer 1: fully connected layers from 768 to 384 features
        self.fc1 = nn.Linear(input_size, hidden_layers[0])
        self.bn1 = nn.BatchNorm1d(hidden_layers[0])

        # Layer 2: fully connected layers from 384 to 256 features
        self.fc2 = nn.Linear(hidden_layers[0], hidden_layers[1])
        self.bn2 = nn.BatchNorm1d(hidden_layers[1])

        # Layer 3: fully connected layers from 256 to 192 features
        self.fc3 = nn.Linear(hidden_layers[1], hidden_layers[2])
        self.bn3 = nn.BatchNorm1d(hidden_layers[2])

        # Layer 4: fully connected layers from 192 to 128 features
        self.fc4 = nn.Linear(hidden_layers[2], hidden_layers[3])
        self.bn4 = nn.BatchNorm1d(hidden_layers[3])

        # Layer 5: fully connected layers from 128 to 96 features
        self.fc5 = nn.Linear(hidden_layers[3], hidden_layers[4])
        self.bn5 = nn.BatchNorm1d(hidden_layers[4])

        # Layer 6: fully connected layers from 96 to 64 features
        self.fc6 = nn.Linear(hidden_layers[4], hidden_layers[5])
        self.bn6 = nn.BatchNorm1d(hidden_layers[5])

        # Layer 7: fully connected layers from 64 to 48 features
        self.fc7 = nn.Linear(hidden_layers[5], hidden_layers[6])
        self.bn7 = nn.BatchNorm1d(hidden_layers[6])

        # Layer 8: fully connected layers from 48 to 32 features
        self.fc8 = nn.Linear(hidden_layers[6], hidden_layers[7])
        self.bn8 = nn.BatchNorm1d(hidden_layers[7])

        # Layer 9: fully connected layers from 32 to 24 features
        self.fc9 = nn.Linear(hidden_layers[7], hidden_layers[8])
        self.bn9 = nn.BatchNorm1d(hidden_layers[8])

        # Layer 10: fully connected layers from 24 to 16 features
        self.fc10 = nn.Linear(hidden_layers[8], hidden_layers[9])
        self.bn10 = nn.BatchNorm1d(hidden_layers[9])

        # Layer 11: fully connected layers from 16 to 8 features
        self.fc11 = nn.Linear(hidden_layers[9], num_classes)

        # ELU activation function: provides fast convergence and robust learning
        self.elu = nn.ELU()

        # Dropout layer: helps prevent overfitting by randomly dropping units during training
        self.dropout = nn.Dropout(dropout)
```

```
def forward(self, x):
    # Generates a tensor of random values from a normal distribution and adds randomness to the data
    x = x + torch.randn_like(x) * 0.1

    # First fully connected layer + batch normalization + ELU activation + dropout
    x = self.fc1(x)
    x = self.bn1(x)
    x = self.elu(x)
    x = self.dropout(x)

    # Second fully connected layer + batch normalization + ELU activation + dropout
    x = self.fc2(x)
    x = self.bn2(x)
    x = self.elu(x)
    x = self.dropout(x)

    # Third fully connected layer + batch normalization + ELU activation + dropout
    x = self.fc3(x)
    x = self.bn3(x)
    x = self.elu(x)
    x = self.dropout(x)

    # Fourth fully connected layer + batch normalization + ELU activation + dropout
    x = self.fc4(x)
    x = self.bn4(x)
    x = self.elu(x)
    x = self.dropout(x)

    # Fifth fully connected layer + batch normalization + ELU activation + dropout
    x = self.fc5(x)
    x = self.bn5(x)
    x = self.elu(x)
    x = self.dropout(x)
```

```
# Sixth fully connected layer + batch normalization + ELU activation + dropout
x = self.fc6(x)
x = self.bn6(x)
x = self.elu(x)
x = self.dropout(x)

# Seventh fully connected layer + batch normalization + ELU activation + dropout
x = self.fc7(x)
x = self.bn7(x)
x = self.elu(x)
x = self.dropout(x)

# Eighth fully connected layer + batch normalization + ELU activation + dropout
x = self.fc8(x)
x = self.bn8(x)
x = self.elu(x)
x = self.dropout(x)

# Ninth fully connected layer + batch normalization + ELU activation + dropout
x = self.fc9(x)
x = self.bn9(x)
x = self.elu(x)
x = self.dropout(x)

# Tenth fully connected layer + batch normalization + ELU activation + dropout
x = self.fc10(x)
x = self.bn10(x)
x = self.elu(x)
x = self.dropout(x)

# Output layer, no activation here
x = self.fc11(x)

return x
```

Script “utils.py”

- Lo **script “utils.py”** contiene diverse funzioni per la valutazione delle prestazioni del modello, la gestione dei file audio, l'estrazione e il preprocessing degli embeddings, e il salvataggio dei dati;
- È utile per preparare i dati e i modelli per l'addestramento e l'analisi, migliorando la gestione e la valutazione dei modelli implementati, che operano su dati audio.

Script “utils.py”

- È composto da nove funzioni:
 1. **compute_metrics;**
 2. **evaluate;**
 3. **get_audio_path;**
 4. **get_genre_idx;**
 5. **get_track_id;**
 6. **load_audio;**
 7. **process_audio;**
 8. **extract_and_preprocess_data;**
 9. **save_data.**

```

import librosa
import numpy as np
import os
import torch

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
from tqdm import tqdm

def compute_metrics(predictions, references):
    acc = accuracy_score(references, predictions)
    precision = precision_score(references, predictions, average='macro')
    recall = recall_score(references, predictions, average='macro')
    f1 = f1_score(references, predictions, average='macro')

    return {
        'accuracy': acc,
        'precision': precision,
        'recall': recall,
        'f1': f1
    }

def evaluate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    predictions = []
    references = []

    with torch.no_grad():
        for i, (x, y) in enumerate(tqdm(dataloader, desc='\nValidation')):
            x, y = x.to(device), y.to(device)
            outputs = model(x)
            loss = criterion(outputs, y)
            running_loss += loss.item()
            pred = torch.argmax(outputs, dim=1)
            predictions.extend(pred.cpu().numpy())
            references.extend(y.cpu().numpy())

    val_metrics = compute_metrics(predictions, references)
    val_metrics['loss'] = running_loss / len(dataloader)
    return val_metrics

def get_audio_path(root, track_id):
    folder = f'{int(track_id) // 1000:03d}'
    return os.path.join(root, folder, f'{int(track_id):06d}.mp3')

def get_genre_idx(metadata, genre_to_idx, idx):
    genre = metadata.iloc[idx]['track.7']
    return genre_to_idx.get(genre, 0)

def get_track_id(metadata, idx):
    return metadata.iloc[idx]['Unnamed: 0']

```

Compute accuracy of the predictions

Compute precision with macro averaging

Compute recall with macro averaging

Compute F1 score with macro averaging

Set the model to evaluation mode

Initialize running loss

List to store model predictions

List to store true labels

Disable gradient calculation for evaluation

Progress bar for validation

Move data to the specified device (CPU or GPU)

Get model predictions

Compute loss using the provided criterion

Accumulate the loss

Get the predicted class labels (with highest score)

Convert predictions to numpy and store

Convert true labels to numpy and store

Calculate evaluation metrics

Compute average loss

Folder structure is based on track ID

Return full path to the audio file

Get the genre from metadata (assuming 'track.7' is the genre column)

Map the genre to an index using genre_to_idx dictionary

Get the track ID from metadata (assuming 'Unnamed: 0' is the track ID column)

```

def load_audio(audio_path, sample_rate):
    try:
        audio, sr = librosa.load(audio_path, sr=sample_rate, mono=True)
    except Exception as e:
        print(f'\nError loading file {audio_path}: {e}')
        return None

    return torch.tensor(audio)

def process_audio(audio, max_length):
    if audio.size(0) > max_length:
        audio = audio[:max_length]
    else:
        padding = max_length - audio.size(0)
        audio = torch.nn.functional.pad(audio, (0, padding))

    return audio

def extract_and_preprocess_data(model, dataloader, device, split):
    model.model.eval()
    embeddings = []
    labels = []

    with torch.no_grad():
        for x, y in tqdm(dataloader, desc=f'\n{split.capitalize()} embeddings and labels extracting'):
            x = x.numpy()
            x = [model.extract(speech=audio_file) for audio_file in x]
            x = np.array(x)
            x = np.mean(x, axis=1)
            embeddings.append(x)
            labels.append(y.numpy())

    embeddings = np.concatenate(embeddings, axis=0)
    labels = np.concatenate(labels, axis=0)

    # Normalize embeddings
    scaler = StandardScaler()
    embeddings = scaler.fit_transform(embeddings)

    return embeddings, labels

def save_data(embeddings, labels, prefix, directory):
    os.makedirs(directory, exist_ok=True)
    np.save(f'{directory}/{prefix}_embeddings.npy', embeddings)
    np.save(f'{directory}/{prefix}_labels.npy', labels)
    print(f'{prefix.capitalize()} embeddings and labels saved in .npy files')

```

Load the audio file in mono format

Print an error message if the file fails to load

Return None if there's an error

Convert the loaded audio to a PyTorch tensor

Truncate the audio to the maximum length

Pad the audio with zeros if it's shorter than max_length

Set the model to evaluation mode

List to store extracted embeddings

List to store corresponding labels

Disable gradient calculation

Convert input data to numpy array

Extract embeddings for each audio file using the model

Convert list of embeddings to numpy array

Average the embeddings along the time axis

Store the embeddings

Store the labels

Concatenate all embeddings

Concatenate all labels

Initialize a standard scaler

Use fit_transform method to normalize the embeddings

Save embeddings as a numpy file

Save labels as a numpy file

Script “train.py”

- Lo **script “train.py”** gestisce un flusso di lavoro completo per l'addestramento, la valutazione e l'ottimizzazione di modelli per la classificazione audio;
- Include la preparazione dei dati e l'addestramento e la valutazione dei modelli, tra cui una SVM creata mediante il classificatore SVC e basata sulle caratteristiche estratte dal modello FNN;
- Fa uso del file di configurazione YAML.

Script “train.py”

- È costituito dal blocco principale, o flusso di esecuzione, e da quattro funzioni:
 1. **train_one_epoch;**
 2. **manage_best_model_and_metrics;**
 3. **manage_early_stopping;**
 4. **load_or_extract_data.**

```

import joblib
import numpy as np
import os
import torch
import torch.nn as nn

from addict import Dict
from data_classes.fma_dataset import FMADataset
from extract_representations.audio_embeddings import AudioEmbeddings
from model_classes.cnn_model import CNNAudioClassifier
from model_classes.fnn_model import FNNAudioClassifier
from sklearn.metrics import hinge_loss
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVC
from torch.utils.data import DataLoader, TensorDataset
from tqdm import tqdm
from utils import compute_metrics, evaluate, extract_and_preprocess_data, save_data
from yaml_config_override import add_arguments

def train_one_epoch(model, dataloader, loss_fn, optimizer, scheduler, device):
    model.train() # Set the model to training mode
    running_loss = 0.0
    predictions = []
    references = []

    # Loop over batches in the dataloader
    for i, (x, y) in enumerate(tqdm(dataloader, desc='\nTraining')):
        x, y = x.to(device), y.to(device) # Move data to the specified device
        optimizer.zero_grad() # Zero the gradients
        outputs = model(x) # Forward pass: compute model predictions
        loss = loss_fn(outputs, y) # Compute the loss
        loss.backward() # Backward pass: compute gradients
        optimizer.step() # Update model weights
        scheduler.step() # Update learning rate if scheduler is defined
        running_loss += loss.item() # Accumulate the loss
        pred = torch.argmax(outputs, dim=1) # Get predicted class labels
        predictions.extend(pred.cpu().numpy()) # Store predictions
        references.extend(y.cpu().numpy()) # Store true labels

    train_metrics = compute_metrics(predictions, references) # Compute training metrics
    train_metrics['loss'] = running_loss / len(dataloader) # Average loss
    return train_metrics # Return computed metrics

def manage_best_model_and_metrics(model, evaluation_metric, val_metrics, best_val_metric, best_model, lower_is_better):
    if lower_is_better:
        is_best = val_metrics[evaluation_metric] < best_val_metric
    else:
        is_best = val_metrics[evaluation_metric] > best_val_metric

    # If the current model is the best, update the best model and metric
    if is_best:
        print(f'New best model found with val {evaluation_metric}: {val_metrics[evaluation_metric]:.4f}')
        best_val_metric = val_metrics[evaluation_metric]
        best_model = model

    return best_val_metric, best_model

```

```

def manage_early_stopping(val_metric, best_val_metric, early_stopping_counter, patience):
    # Check if the validation metric has improved
    if val_metric <= best_val_metric:
        early_stopping_counter = 0        # Reset counter if improvement is seen
    else:
        early_stopping_counter += 1      # Increment counter if no improvement

    # If the counter exceeds patience, trigger early stopping
    if early_stopping_counter >= patience:
        print("Early stopping triggered.")
        return True

    return False

def load_or_extract_data(root, metadata_file, sample_rate, max_duration, batch_size, device, directory, split):
    # Define file paths for embeddings and labels
    embedding_file = f'{directory}/{split}_embeddings.npy'
    label_file = f'{directory}/{split}_labels.npy'

    # Check if the embeddings and labels already exist as .npy files
    if os.path.exists(embedding_file) and os.path.exists(label_file):
        # Load pre-saved embeddings and labels
        embeddings = np.load(embedding_file)
        labels = np.load(label_file)
    else:
        model = AudioEmbeddings()        # Initialize the audio embedding extraction model

        # Create the dataset and dataloader for the specified split
        dataset = FMADataset(root, metadata_file, split, sample_rate, max_duration)
        dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=(split == 'training'))

        # Extract embeddings and labels from the audio data
        embeddings, labels = extract_and_preprocess_data(model, dataloader, device, split)

        # Save the extracted embeddings and labels as .npy files for future use
        save_data(embeddings, labels, split, directory)

    return embeddings, labels            # Return the embeddings and labels

```

```

if __name__ == '__main__':
    config = Dict(add_arguments())          # Load configuration from YAML file
    device = torch.device('cuda' if torch.cuda.is_available() and config.training.device == 'cuda' else 'cpu')

    # Check if training embeddings and labels already exist and upload them, otherwise extract and save them
    train_emb, train_lab = load_or_extract_data(config.data.dataset_dir,
                                                config.data.metadata_file,
                                                config.training.sample_rate,
                                                config.training.max_duration,
                                                config.training.batch_size,
                                                config.training.device,
                                                config.training.npy_dir,
                                                config.split.train)

    # Check if validation embeddings and labels already exist and upload them, otherwise extract and save them
    val_emb, val_lab = load_or_extract_data(config.data.dataset_dir,
                                            config.data.metadata_file,
                                            config.training.sample_rate,
                                            config.training.max_duration,
                                            config.training.batch_size,
                                            config.training.device,
                                            config.training.npy_dir,
                                            config.split.val)

    # Convert embeddings and labels to tensors
    train_emb_tensor = torch.tensor(train_emb, dtype=torch.float32)
    train_lab_tensor = torch.tensor(train_lab, dtype=torch.long)
    val_emb_tensor = torch.tensor(val_emb, dtype=torch.float32)
    val_lab_tensor = torch.tensor(val_lab, dtype=torch.long)

    # Create TensorDatasets for training and validation sets
    train_ds = TensorDataset(train_emb_tensor, train_lab_tensor)
    val_ds = TensorDataset(val_emb_tensor, val_lab_tensor)

    # Create DataLoaders for the datasets
    train_dl = DataLoader(train_ds, config.training.batch_size, shuffle=True)
    val_dl = DataLoader(val_ds, config.training.batch_size, shuffle=False)

    # Instantiate the CNN and FNN models
    cnn_model = CNNAudioClassifier(config.model.input_size,
                                   config.model.num_classes,
                                   config.model.dropout).to(device)
    fnn_model = FNNAudioClassifier(config.model.input_size,
                                   config.model.hidden_layers,
                                   config.model.num_classes,
                                   config.model.dropout).to(device)

    cnn_model.to(device)
    fnn_model.to(device)

```

```

# Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
cnn_optimizer = torch.optim.Adam(cnn_model.parameters(), lr=config.training.learning_rate,
                                  weight_decay=config.training.weight_decay)
fnn_optimizer = torch.optim.Adam(fnn_model.parameters(), lr=config.training.learning_rate,
                                  weight_decay=config.training.weight_decay)

# Learning rate scheduler for warmup and decay
total_steps = len(train_dl) * config.training.epochs
warmup_steps = int(total_steps * config.training.warmup_ratio)

# Warmup + linear decay schedule
scheduler_lambda = lambda step: (step / warmup_steps) if step < warmup_steps
else max(0.0, (total_steps - step) / (total_steps - warmup_steps))
cnn_scheduler = torch.optim.lr_scheduler.LambdaLR(cnn_optimizer, lr_lambda=scheduler_lambda)
fnn_scheduler = torch.optim.lr_scheduler.LambdaLR(fnn_optimizer, lr_lambda=scheduler_lambda)

# List of models to train
models = [('CNN', cnn_model, cnn_optimizer, cnn_scheduler, config.best.cnn),
          ('FNN', fnn_model, fnn_optimizer, fnn_scheduler, config.best.ff)]

# Initialization of early stopping parameters
patience = 3
early_stopping_counter = 0

for model_name, model, optimizer, scheduler, best_model in models:
    print(f'\n\nStart {model_name} model training and validation')

    # Initialize best validation metric and model
    best_val_metric = float('-inf') if config.training.best_metric_lower_is_better else float('-inf')
    best_model = None

    for epoch in range(config.training.epochs):
        print(f'\nEpoch {epoch+1}/{config.training.epochs}')

        # Train for one epoch and evaluate on validation set
        train_metrics = train_one_epoch(model, train_dl, criterion, optimizer, scheduler, device)
        val_metrics = evaluate(model, val_dl, criterion, device)
        print(f'Train loss: {train_metrics["loss"]:.4f} - Train accuracy: {train_metrics["accuracy"]:.4f}')
        print(f'Val loss: {val_metrics["loss"]:.4f} - Val accuracy: {val_metrics["accuracy"]:.4f}')

        # Update best model if validation performance improves
        best_val_metric, best_model = manage_best_model_and_metrics(
            model,
            config.training.evaluation_metric,
            val_metrics,
            best_val_metric,
            best_model,
            config.training.best_metric_lower_is_better
        )

        # Check early stopping conditions
        if manage_early_stopping(val_metrics['loss'], best_val_metric, early_stopping_counter, patience):
            break        # Stop training if early stopping is triggered

    # Save the best model
    os.makedirs(config.training.checkpoint_dir, exist_ok=True)
    torch.save(best_model.state_dict(), f'{config.training.checkpoint_dir}/best_{model_name.lower()}_model.pt')
    print('Model saved.')

```

```

print(f'\n\nStart SVM model grid search and validation\n')
svm_model = SVC() # Instantiate the SVM model

# After training CNN and FNN models, use the FNN model to extract features on training and validation sets for SVM
fnn_model.eval()
with torch.no_grad():
    train_features = fnn_model(torch.tensor(train_emb, dtype=torch.float32).to(device)).cpu().numpy()
    val_features = fnn_model(torch.tensor(val_emb, dtype=torch.float32).to(device)).cpu().numpy()

# Parameter distribution for SVM randomized search
param_dist = {
    'C': [0.1, 1, 10, 100], # Regularization parameter
    'gamma': ['scale', 'auto'], # Kernel coefficient for 'rbf'
    'kernel': ['linear', 'rbf'] # Kernel type
}

# Randomized search across the specified hyperparameter distributions
random_search = RandomizedSearchCV(svm_model, param_dist, n_iter=10, cv=5, scoring='accuracy', verbose=3)
for _ in tqdm(range(1)):
    random_search.fit(train_features, train_lab)
best_svm = random_search.best_estimator_
best_parameters = random_search.best_params_
print(f'\nBest SVM: {best_parameters}')

# Save the best SVM model
joblib.dump(best_svm, f'{config.training.checkpoint_dir}/{config.best.svm}')
print('Model saved.')

# Evaluate SVM on validation set
val_predictions = best_svm.predict(val_features)
svm_val_metrics = compute_metrics(val_predictions, val_lab)
decision_scores = best_svm.decision_function(val_features)
hinge_loss_value = hinge_loss(val_lab, decision_scores)
print(f'\nVal loss: {hinge_loss_value:.4f} - Val accuracy: {svm_val_metrics["accuracy"]:.4f}')

```

Script “test.py”

- Lo **script “test.py”** è progettato per gestire il caricamento dei modelli pre-addestrati, l'acquisizione dei dati di test, e la valutazione finale delle prestazioni dei modelli sui dati di test;
- È utile per ottenere una valutazione finale e stampare le metriche di prestazione dei modelli addestrati;
- È formato anch'esso dal blocco principale o flusso di esecuzione.

[illegible]

```

# Load the pre-trained model weights for the CNN, FNN and SVM models
cnn_model.load_state_dict(torch.load(f'{config.training.checkpoint_dir}/{config.best.cnn}', weights_only=True))
print('CNN Model loaded.')
fnn_model.load_state_dict(torch.load(f'{config.training.checkpoint_dir}/{config.best.ff}', weights_only=True))
print('FNN Model loaded.')
svm_model = joblib.load(f'{config.training.checkpoint_dir}/{config.best.svm}')
print('SVM Model loaded.')

criterion = nn.CrossEntropyLoss() # Define the loss function for evaluation
models = [('CNN', cnn_model), ('FNN', fnn_model)] # List of models to evaluate

# Evaluate each model on the test dataset
for model_name, model in models:
    test_metrics = evaluate(model, test_dl, criterion, device) # Evaluate the model on the test dataset

    # Print out the evaluation metrics for each model
    for key, value in test_metrics.items():
        print(f'{model_name} Model Test {key}: {value:.4f}')

# After evaluate CNN and FNN models, use the FNN model to extract features on test set for SVM
fnn_model.eval()
with torch.no_grad():
    test_features = fnn_model(test_emb_tensor.to(device)).cpu().numpy()

# Evaluate SVM on test set
test_predictions = svm_model.predict(test_features)
svm_test_metrics = compute_metrics(test_predictions, test_lab)
decision_scores = svm_model.decision_function(test_features)
hinge_loss_value = hinge_loss(test_lab, decision_scores)
for key, value in svm_test_metrics.items():
    print(f'\nSVM Model Test {key}: {value:.4f}', end='')
print(f'\nSVM Model Test loss: {hinge_loss_value:.4f}')

```

Altri file del progetto

- Il progetto in questione è costituito anche dai seguenti file:
 1. **.gitignore;**
 2. **LICENSE;**
 3. **prepare.sh;**
 4. **README.md;**
 5. **requirements.txt;**
 6. **Sei file di estensione “.npz”;**
 7. **Sei immagini di estensione “.png”.**

Descrizione dei file

- Il file “**.gitignore**” contiene i modelli e i percorsi che Git dovrebbe ignorare;
- Il file “**LICENSE**” definisce la licenza del progetto, specificando i termini sotto i quali il software può essere utilizzato, modificato e distribuito. La licenza può variare da permissiva, come MIT o Apache, a più restrittiva, come GPL. La licenza applicata in questo progetto è MIT.

Descrizione dei file

- Il file “**prepare.sh**” è uno script Bash che viene usato per preparare l'ambiente di lavoro del progetto. In questo caso, il suddetto file è utilizzato per scaricare il dataset e i metadati, estrarli in una specifica cartella e installare le librerie del file “requirements.txt”;
- Il file “**README.md**” è un documento di testo scritto in Markdown che fornisce una panoramica del progetto.

Descrizione dei file

- Il file “**requirements.txt**” elenca tutte le dipendenze Python, ossia le librerie necessarie per eseguire il progetto;
- I sei file di estensione “**.npz**” sono:
 1.  `test_embeddings.npz`
 2.  `test_labels.npz`
 3.  `training_embeddings.npz`
 4.  `training_labels.npz`
 5.  `validation_embeddings.npz`
 6.  `validation_labels.npz`

Descrizione dei file

- Questi file contengono gli embeddings estratti dai file audio e normalizzati, e le relative labels, salvate in appositi file per accelerare il processo di addestramento della rete;
- Infatti, l'estrazione di embeddings e labels per i file di train ha richiesto 29 ore!
- Più veloce è stata, invece, l'estrazione dei medesimi dati per i file di validation e test, che ha richiesto circa 4 ore a testa.

Descrizione dei file

- Le **sei immagini di estensione “.png”** contengono i risultati ottenuti durante le fasi di addestramento, validazione e test dei tre modelli realizzati. Esse sono:

1.  **cnn_test.png**
2.  **cnn_training_and_validation.png**
3.  **fnn_test.png**
4.  **fnn_training_and_validation.png**
5.  **svm_test.png**
6.  **svm_validation.png**

Risultati in fase di train e validation

Training embeddings extracting: 100%|██████████| 800/800 [28:59:40<00:00, 130.48s/it]

Validation embeddings extracting: 100%|██████████| 100/100 [4:03:56<00:00, 146.37s/it]

Test embeddings extracting: 100%|██████████| 100/100 [3:48:50<00:00, 137.31s/it]

CNN

Epoch 100/100

Train loss: 0.8036 - Train accuracy: 0.7477

Val loss: 1.3841 - Val accuracy: 0.5962

Model saved.

FF

Epoch 100/100

Train loss: 0.9612 - Train accuracy: 0.6852

Val loss: 1.2600 - Val accuracy: 0.5850

Model saved.

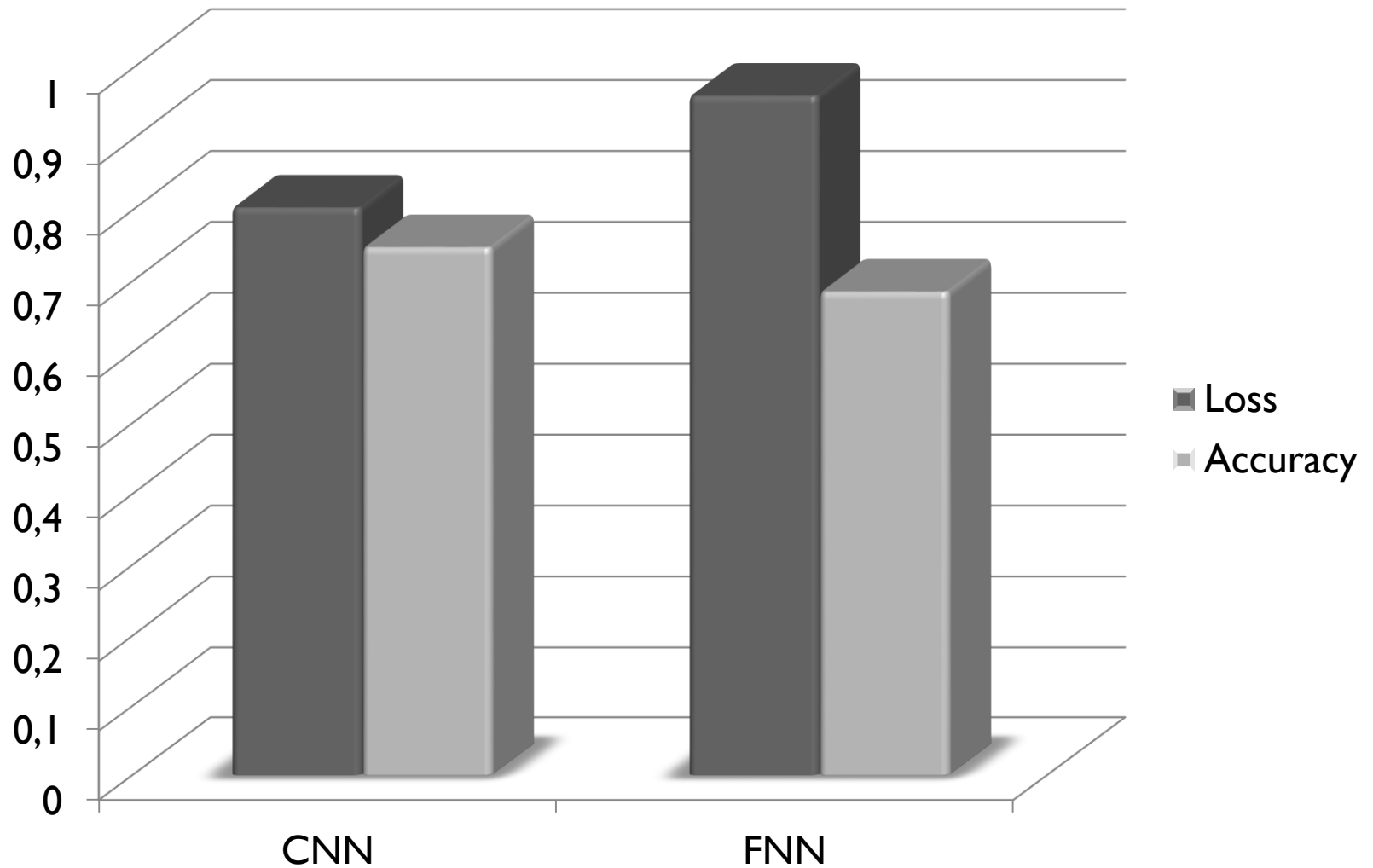
SVM

Best SVM: {'kernel': 'rbf', 'gamma': 'auto', 'C': 1}

Model saved.

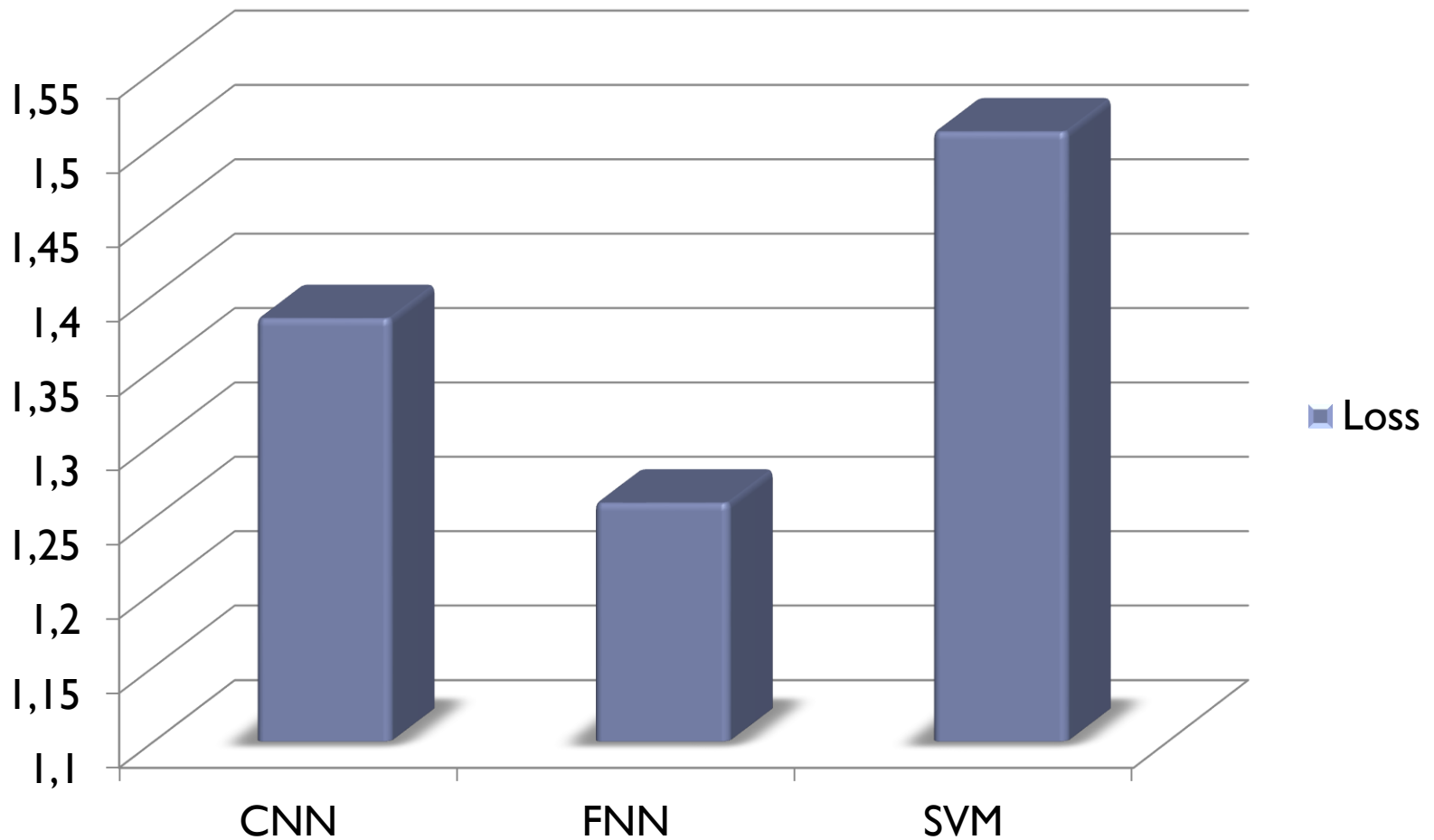
Val loss: 1.5096 - Val accuracy: 0.5837

Risultati in fase di train



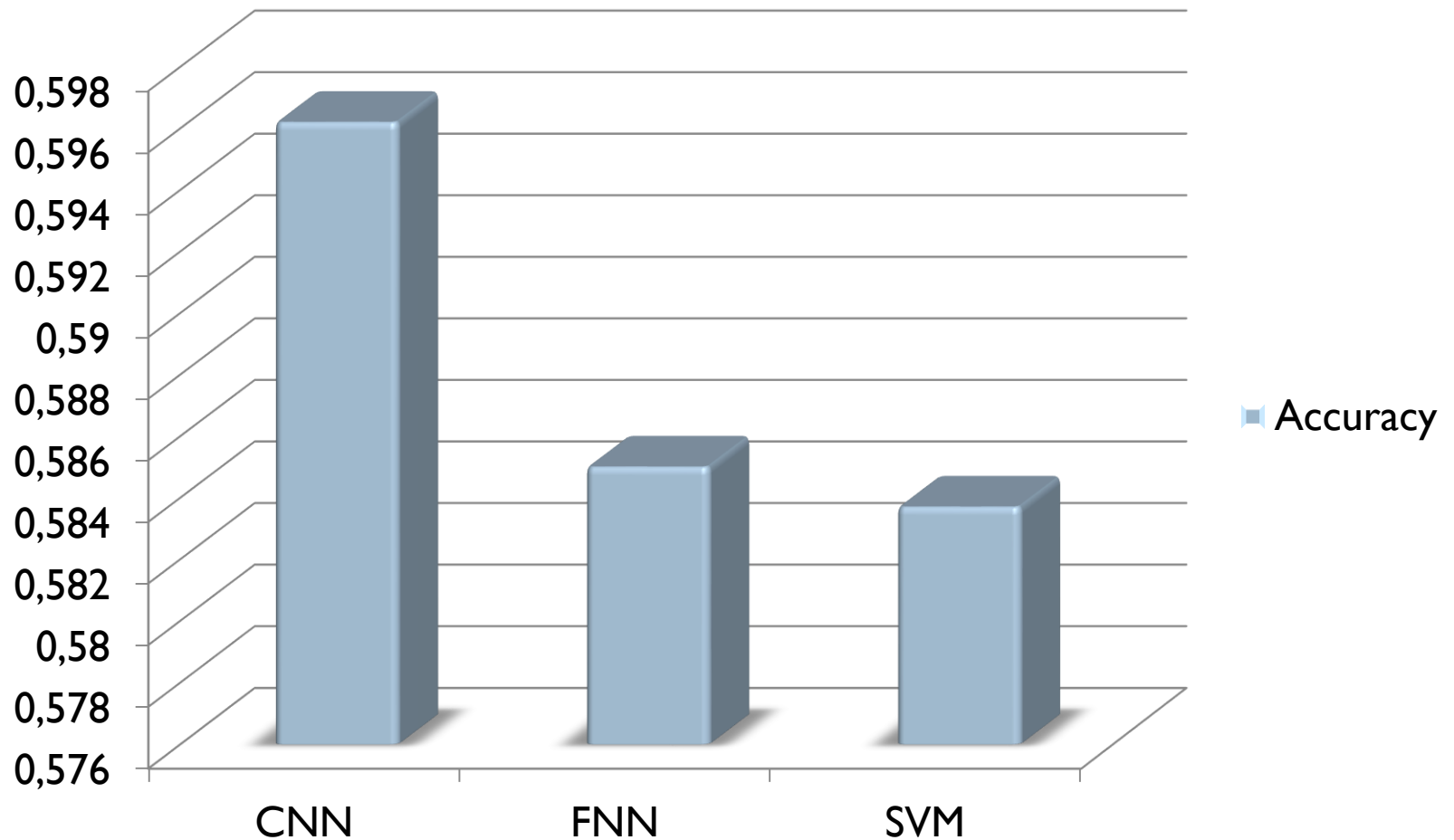
Risultati in fase di validation

Loss



Risultati in fase di validation

Accuracy



Risultati in fase di test

CNN Model Test accuracy: 0.5212

CNN Model Test precision: 0.5476

CNN Model Test recall: 0.5212

CNN Model Test f1: 0.5256

CNN Model Test loss: 1.5867

FNN Model Test accuracy: 0.5288

FNN Model Test precision: 0.5196

FNN Model Test recall: 0.5287

FNN Model Test f1: 0.5212

FNN Model Test loss: 1.3650

SVM Model Test accuracy: 0.5413

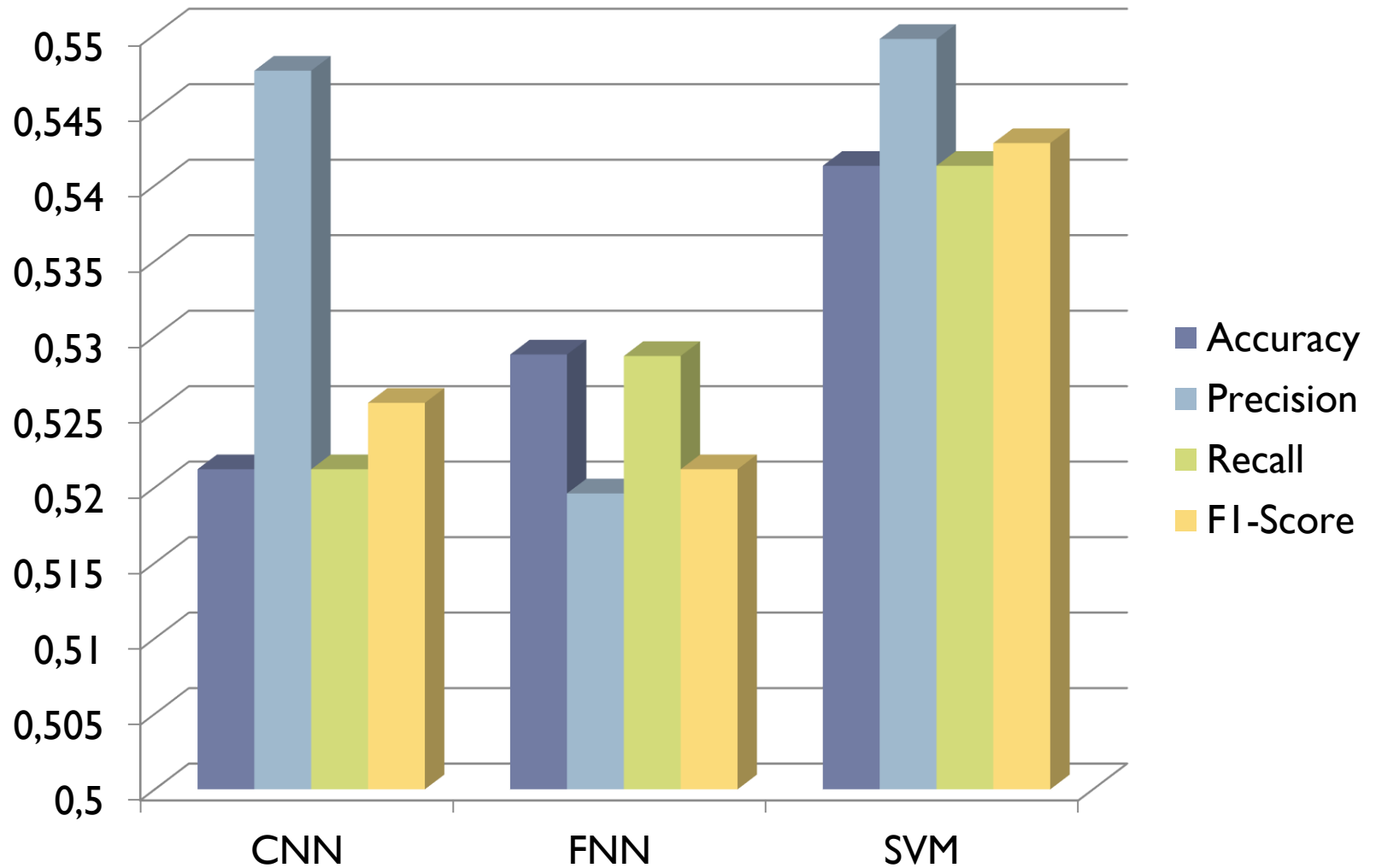
SVM Model Test precision: 0.5497

SVM Model Test recall: 0.5413

SVM Model Test f1: 0.5428

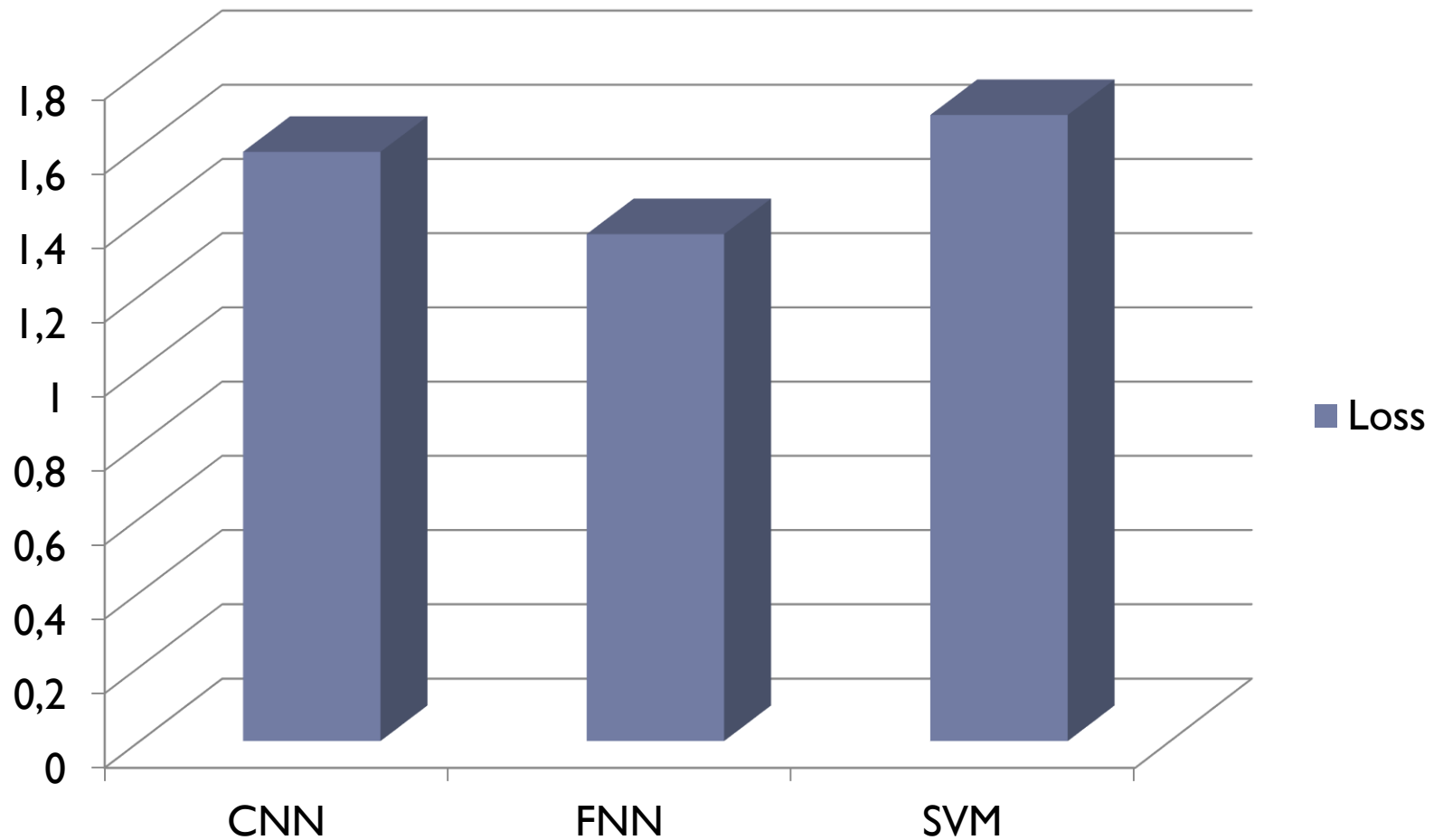
SVM Model Test loss: 1.6860

Risultati in fase di test



Risultati in fase di test

Loss



Link utili

1. Repository GitHub:

- <https://github.com/LuigiPioFaletra/Machine-Learning-Project>;

2. FMA (Free Music Archive):

- <https://github.com/mdeff/fma>;

3. Scaricare dataset e metadati:

- https://os.unil.cloud.switch.ch/fma/fma_small.zip;
- https://os.unil.cloud.switch.ch/fma/fma_metadata.zip.